TrustVault: Trustless, Scalable, and Low-Cost On-Chain Academic Credential Verification Using Internet Computer Protocol

Dr. Shruti Jadon
Department of Computer Science
PES University
Bengaluru, Karnataka, India
shrutijadon@pes.edu

Varun Gudamsetti
Department of Computer Science
PES University
Bengaluru, Karnataka, India
varun.techmail@gmail.com

***Abstract*—Academic credential fraud imposes significant costs on institutions and employers worldwide. Existing solutions rely on either centralized registries or permissioned blockchains, both of which retain trust dependencies and single points of failure. This paper presents *TrustVault*, a fully on-chain academic credential verification system deployed on the Internet Computer Protocol (ICP) mainnet (canister `pekzw-diaaa-aaaad-afh3q-cai`). TrustVault stores credential data and its serving frontend inside ICP canisters and uses ICP *Certified Variables* — backed by threshold BLS signatures from a decentralized subnet — for mathematically verifiable, zero-trust authentication. An incremental binary Merkle tree provides O($\log n$) update cost as the credential database scales, with each leaf representing a cryptographic commitment to a certificate record. Live mainnet benchmarks show issuance latency of ≈1.9 s, verification queries as fast as 60 ms, parallel throughput up to 7.3 certs/s, concurrent mixed-load throughput of 13.1 ops/s, and issuance cost of ≈$0.00002 per certificate — over 10,000× cheaper than comparable Ethereum operations. The prototype demonstrates practical feasibility; production deployment requires cryptographic hash hardening.**

***Index Terms*—Internet Computer Protocol, Academic Credential Verification, Certified Variables, Merkle Tree, Zero-Trust, Motoko, Threshold BLS Signatures**

I. Introduction

Academic credential fraud is a well-documented, escalating problem: studies estimate that a significant fraction of professional job applications misrepresent qualifications [1], and traditional verification — contacting institutions manually — is slow, expensive, and centralized. Blockchain-based alternatives have been proposed but each carries practical limitations. Bitcoin and Ethereum anchor credential hashes at prohibitive gas fees (tens of dollars per write), and the verification UI remains on a central server. Permissioned ledgers such as Hyperledger Fabric reduce cost but reintroduce consortium trust [14]. Hybrid systems storing only hashes on-chain with data off-chain restore centralization through the back door.

The *Internet Computer Protocol* (ICP) [2] offers a qualitatively different model. ICP runs smart contracts called *canisters* — WebAssembly modules replicated across independent *subnets* of geographically diverse nodes. Consensus achieves finality in ≈1–3 s via a BFT notarization protocol backed by the *Random Beacon* (threshold BLS over previous round outputs). Two protocol features are critical to this work: *on-chain web serving* (canisters respond to HTTP, eliminating cloud hosting) and *Certified Variables* (a native API where a canister commits a 32-byte value that the subnet collectively signs with a threshold BLS key [3], enabling data verification with no trust in the canister or any single node). Query calls — read-only operations bypassing consensus — return in <500 ms at no cost to the caller.

Contributions:

- TrustVault — among the first fully on-chain academic credential systems (frontend *and* backend) deployed on a public blockchain mainnet, demonstrated with live mainnet measurements.
- An incremental Merkle tree with O($\log n$) update cost, integrated with Certified Variables for trustless proof generation.
- A rigorous live mainnet evaluation: issuance, verification, throughput, finality, and concurrent mixed-load.
- A cross-platform comparison against Ethereum PoS [12] and Hyperledger Fabric [14] using only published benchmarks.

II. System Design

A. Architecture

The Motoko backend is organized into five modules: **main.mo** (public actor — `issueCertificate`, `revokeCertificate`, `verifyCertificate`), **CertificateManager.mo** (record hashing and management), **MerkleTree.mo** (incremental binary Merkle tree), **Utils.mo** (hex encoding, hash combination), and **PerformanceMetrics.mo** (telemetry). The frontend is a React + Vite single-page application inside an ICP asset canister, served directly over HTTPS from the blockchain with no external hosting. It communicates with the backend through auto-generated Candid type bindings. Fig. 1 illustrates the full system.

Access control is enforced via ICP *Principals* — Ed25519-derived cryptographic identities. Only registered

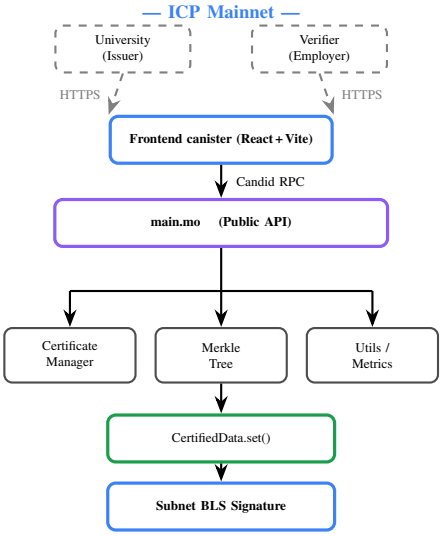

Fig. 1. TrustVault system architecture. Universities and verifiers connect via HTTPS. The frontend routes requests to the backend canister, which issues and verifies certificates using a Merkle tree whose root is BLS-signed by the ICP subnet.

university principals may call `issueCertificate` or `revokeCertificate`; `verifyCertificate` is a free public query. Each certificate (Table I) carries a certificate hash of its core identity fields (SHA-256 in production; see Section V for prototype limitations) and a consensus-set nanosecond timestamp. Stable Motoko variables persist the certificate map and Merkle root across canister upgrades.

TABLE I
CERTIFICATE SCHEMA

Field	Type	Description
`certificate_hash`	Text	UTF-8 hex of core fields†
`issuer`	Record	University name, canister ID
`recipient`	Record	Student name, ID, principal
`credential`	Record	Degree, major, GPA
`is_revoked`	Bool	Revocation flag
`block_timestamp`	Int	Consensus timestamp (ns)

† Prototype; SHA-256 required for production (see Section V).

B. Incremental Merkle Tree and Certified Data

All certificate hashes are leaves of a complete binary Merkle tree stored as a flat array of size $2C-1$, where C is the smallest power of two $\geq$ the certificate count. Leaf i is at index $C-1+i$; the root is at index 0. Each internal node stores the combined hash of its two children:

$$\texttt{nodes}[j] = \texttt{combineHashes}(\texttt{nodes}[2j+1],\ \texttt{nodes}[2j+2]) \quad (1)$$

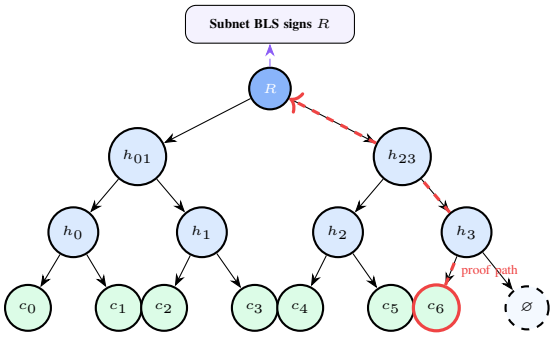

Fig. 2. Merkle tree used for certificate verification. Adding leaf c_6 (red) requires updating only three hashes to reach root R, which the ICP subnet BLS-signs each consensus round.

(XOR-based in the current prototype; SHA-256 in production — see Section V.) Inserting a new leaf recomputes only the $O(\log n)$ path to the root (Fig. 2). After each issuance, the 32-byte Merkle root is committed with `CertifiedData.set()`, and the subnet threshold BLS-signs it within the next consensus round. Revocation triggers a full O(n) rebuild; the new certified root invalidates all pre-revocation proofs within one consensus round (≈2 s).

III. Trustless Verification

Verification requires no trust in the issuing institution. Given a certificate and its Merkle proof π, the verifier: (1) recomputes $h_c = H(\text{cert fields})$ and checks it matches the stored `certificate_hash` (H is SHA-256 in production; the prototype substitutes UTF-8 hex encoding — see Section V); (2) walks the Merkle proof to obtain the claimed root $\hat{R} = \text{verifyProof}(h_c, \pi)$; (3) fetches the ICP *Certificate* (authenticated state tree + BLS witness) from any ICP gateway; (4) verifies the subnet's threshold BLS signature over the state tree root using the publicly known subnet key; (5) confirms the canister's certified value in the state tree equals $\hat{R}$.

The complete chain of trust is:

$$\text{cert fields} \xrightarrow{H} h_c \xrightarrow{\pi} \hat{R} \xrightarrow{\text{state tree}} \text{BLS sig} \xrightarrow{\text{subnet key}} \checkmark \quad (2)$$

Steps 1–2 execute client-side with no network call. Step 3 requires a single HTTP query to any ICP gateway. The subnet public key is published by the DFINITY Foundation and independently verifiable via the on-chain NNS — no step requires contacting the issuing institution.

IV. Performance Evaluation

All benchmarks were run against the live ICP mainnet canister `pekzw-diaaa-aaaad-afh3q-cai` in March 2026 using a Node.js benchmark suite (`@dfinity/agent`) from India (≈50–80 ms RTT to ICP boundary nodes). Each metric was collected over 3 trials; we report min, mean, p50, p95, and p99.

TABLE II
PARALLEL ISSUANCE BENCHMARK — ICP MAINNET (3 TRIALS EACH)

Batch (n)	Wall mean (ms)	p95 (ms)	Thpt (cps)	Stddev
1	1943	2111	0.52	143.6
10	2596	3054	4.02	498.1
50	6933	7320	7.23	325.2
100	13969	15416	7.31	1926.5

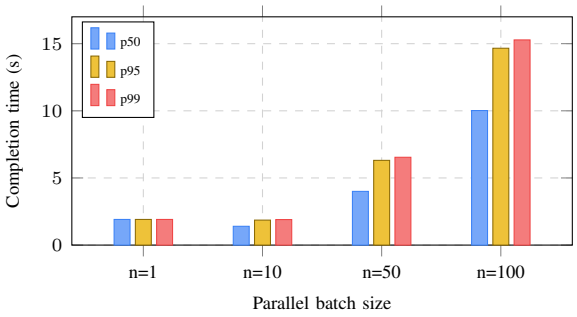

Fig. 3. Parallel issuance completion time (p50/p95/p99) by batch size. Each certificate takes ≈1.9 s; the rising bars at n=100 reflect the last certificate waiting behind 99 others in the queue, not a slower per-cert rate. Peak throughput: 7.31 certs/s.

TABLE III
VERIFICATION LATENCY — ICP MAINNET, 500-CERT DATABASE

Queries	Min	Mean	p50	p75	p95	p99
1	60	227	202	310	396	414
10	60	359	345	460	600	627
100	—	405	400	505	693	937

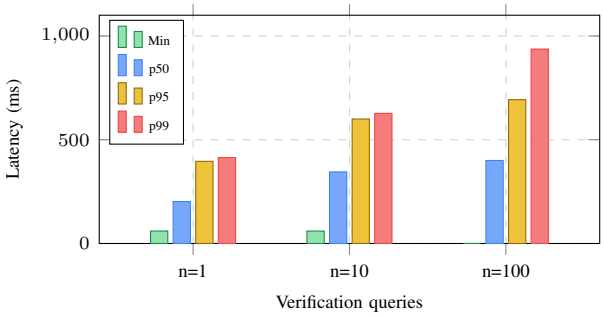

Fig. 4. Verification latency for 1, 10, and 100 queries against a 500-cert database. p99 stays below 1 s. Min bar is omitted for n=100 due to a measurement artifact.

A. Issuance Latency

Table II reports wall time and throughput for parallel issuance batches. At n=1, wall time mirrors one consensus round (≈1.9 s). Parallelism amortizes the consensus overhead: at n=50, throughput reaches 7.23 certs/s. Fig. 3 shows *completion time by queue position*. Each individual certificate takes ≈1.9 s to process regardless of batch size — the rising bars at n=100 do not mean a single certificate takes 15 s. Rather, since all n certs are submitted simultaneously but the canister serialises update calls one-per-consensus-round, the k-th cert in line finishes only after the $k-1$ ahead of it complete. At n=100, the 50th cert is done at ≈10 s and the 99th at ≈15 s from submission — matching the overall wall-time of 13,969 ms in Table II.

B. Verification Latency

Verification is a *query* call — no consensus, no fee. Table III and Fig. 4 report latency over a live database of 500 certificates. Minimum observed latency was **60 ms**, confirming that Merkle proof computation adds negligible overhead. The p99 remains below 938 ms even for 100-query bursts, making it suitable for real-time employer verification portals.

C. Throughput and Scalability

Fig. 5 overlays issuance throughput under two conditions: a fresh (low-count) database and a large database (≈3,280–3,461 existing certificates). The two curves nearly coincide, confirming O($\log n$) tree-insert scalability independent of database size. Peak throughput of **7.3 certs/s** is achieved at batch size 50. Over a 30-second sequential run, sustained throughput was stable at 0.50–0.60 certs/s with no degradation. A peak burst of 100 simultaneous certificates completed at 6.69 certs/s with **0% error rate**.

D. Concurrent Mixed-Load

The concurrent benchmark submits interleaved issuance and verification calls at concurrency level $c \in \{1, 5, 10, 25, 50\}$. Fig. 6 shows total throughput (ops/s, counting both call types) growing from 0.48 at c=1 to **13.09 ops/s** at c=50 — a ≈27× speed-up with **zero errors** at all levels. Fig. 7 shows verification latency (query, no consensus) remains stable in the 554–635 ms range across all concurrency levels, while issuance latency grows gracefully from 2.2 s to 2.9 s.

E. Finality Time and Cross-Platform Comparison

Block finality was measured over 20 independent update calls: min 1,443 ms, mean **2,400 ms**, stddev 385 ms, p50 2,490 ms, p75 2,564 ms, p95 2,987 ms, p99 **3,047 ms** — a tight worst-case bound suitable for interactive applications. Table IV places these results against published benchmarks for Ethereum PoS and Hyperledger Fabric; all non-ICP figures are from cited literature. Ethereum finality (≈24 s) is specified in [12]; network throughput (15–30 TPS) is from BLOCKBENCH [13]; gas costs for a credential write are derived from the Ethereum Yellow Paper [11]; Fabric throughput (≈3,500 TPS) is from Androulaki et al. [14]. ICP query calls (verification) are free to the caller; only update calls (issuance) consume cycles [3].

V. SECURITY ANALYSIS

A. Threat Model

We consider the following adversarial capabilities: (i) a *malicious issuer* who attempts to forge or alter credentials without authorisation; (ii) a *network adversary* who intercepts or

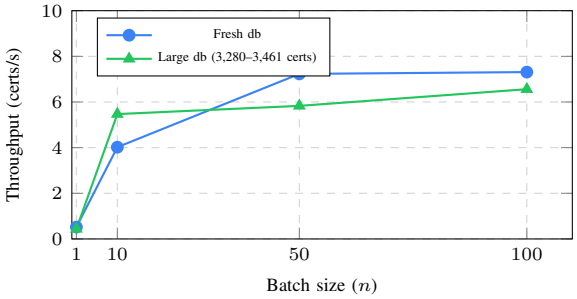

Fig. 5. Issuance throughput vs. batch size for a fresh and a large database. Near-identical curves confirm O($\log n$) Merkle insert cost regardless of database size.

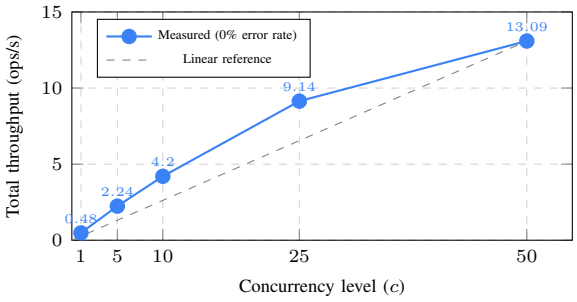

Fig. 6. Mixed-load throughput (30% issue, 70% verify) at increasing concurrency levels. Sub-linear growth is because issuance calls are serialised per consensus round.

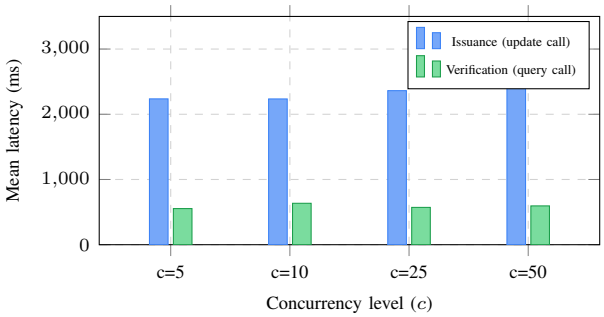

Fig. 7. Mean latency for issuance and verification at each concurrency level. Verification is ≈3× faster than issuance since it requires no consensus.

TABLE IV
CROSS-PLATFORM COMPARISON (NON-ICP VALUES FROM CITED LITERATURE)

Metric	ICP (TrustVault)	Ethereum PoS	H. Fabric
Finality	**2.40 s** (meas.)	≈24 s [12]	<1 s [14]
Throughput	7.3 certs/s	15–30 TPS [13]	≈3500 TPS [14]
Issuance cost	≈**$0.00002** (est.†)	$0.20–$2.00 [11]	Near-zero
Verification cost	**$0** (query)	$0.05–$0.50 [11]	Near-zero
Frontend on-chain	**Yes**	No	No
Trust model	**Zero-trust**	Zero-trust	Permissioned

modifies traffic between client and node; (iii) a *compromised node subset* in which up to $f < n/3$ subnet replicas behave arbitrarily (Byzantine); (iv) a *replay attacker* who reuses a valid certificate or submits duplicate issuance requests; and (v) a *canister manipulator* who attempts to alter certified state without detection. We assume SHA-256 is collision-resistant and BLS12-381 is secure under the Computational Diffie–Hellman (CDH) assumption [6].

B. Security Guarantees

Note on prototype implementation: The security properties below describe the system as designed for production, assuming a collision-resistant hash function (SHA-256) for both certificate hashing and Merkle tree node combination. The current prototype uses simplified placeholder functions (UTF-8 hex encoding for certificate hashes; XOR-based combination for Merkle nodes). These do not provide cryptographic collision resistance; a production deployment must replace them with SHA-256 before the guarantees below hold.

Data Integrity. Any modification to a stored certificate changes its SHA-256 hash h_c. The Merkle proof computed from the original hash no longer validates; the verifier detects the mismatch in Step 1 of the verification protocol (Section III) without any network call. *Backed by: Merkle tree + SHA-256 hash consistency.*

Authenticity. The Merkle root is threshold BLS-signed by the ICP subnet. Forging a valid subnet signature requires compromising $> 1/3$ of subnet nodes simultaneously — computationally and logistically infeasible across a geographically diverse, independently operated network. No single node can unilaterally produce a fraudulent signature. *Backed by: ICP Certified Variables + BFT threshold BLS.*

Replay Resistance. Each certificate carries a unique `certificate_id` and a consensus-set nanosecond `block_timestamp`. Duplicate issuance attempts with the same ID are rejected at the application layer before any state change. *Backed by: unique ID enforcement + consensus timestamp.*

Unauthorized Access. `issueCertificate` is gated by registered Ed25519-derived ICP Principals. An attacker without a registered university key receives an "Unauthorized" error before any state change. Even a colluding replica node cannot bypass this check, since access control executes within the replicated WebAssembly runtime. *Backed by: Ed25519 Principal registry.*

Revocation Freshness. Revocation triggers an O(n) Merkle rebuild and a new `CertifiedData.set()` call. The updated root is BLS-signed within one consensus round (≈2 s), invalidating all pre-revocation Merkle proofs far faster than traditional CRL refresh cycles.

Frontend Integrity. ICP's certified HTTP mechanism signs the byte-level content of canister asset responses. A MITM who modifies the served JavaScript produces a response body that mismatches the certified HTTP header signature, making tampering detectable in transit.

C. Trust Assumptions and Limitations

TrustVault's guarantees rest on: (1) SHA-256 collision resistance; (2) BLS12-381 CDH security; (3) ICP honest-majority subnet ($< n/3$ malicious replicas); and (4) NNS governance integrity for subnet key management. Revocation requires an O(n) Merkle rebuild; for large databases this adds measurable latency. The system is ICP-specific with no cross-chain verification support.

Prototype limitations for production hardening: (a) *Hashing*: the current implementation uses simplified placeholder hash functions that do not satisfy collision resistance; SHA-256 must be substituted before the security guarantees in Section V apply. (b) *University registration*: for ease of prototype testing, `registerUniversity` is a permissionless call that any ICP principal can invoke. This is an intentional simplification for development convenience; a production deployment would restrict registration to the canister controller or an NNS-governed whitelist with appropriate identity validation.

VI. Related Work and Conclusion

Related Work. Blockcerts [7] anchors diploma hashes on Bitcoin, but verification still requires the issuer's web server and the UI is centralized. EduCTX [8] uses a permissioned Ark-based blockchain and requires trust in the university consortium. Ethereum-based systems [9] store only hashes on-chain, keeping full data off-chain and restoring centralization; gas costs [11] vary significantly post-EIP-1559 and post-Merge, often exceeding \$1–\$5 per credential write under high network demand. W3C Verifiable Credentials [10] is a data model, not a storage platform; most implementations use permissioned ledgers or central registries. Self-Sovereign Identity systems [16] target individual data control but do not specify on-chain storage or serving. None of the above systems (a) store the full credential record on a public blockchain, (b) serve the verification UI from the blockchain, or (c) provide reproducible mainnet performance benchmarks. Note that Table IV compares TrustVault's credential-specific throughput against Fabric's general-transaction TPS [14] — not a matched workload comparison; Fabric's figure reflects a permissioned setting with no credential-specific overhead.

Conclusion. TrustVault demonstrates that ICP's Certified Variables and on-chain web serving enable a practical, fully trustless academic credential system. Live mainnet benchmarks confirm: issuance in ≈1.9 s; verification in 60–937 ms; parallel throughput of 7.3 certs/s; concurrent mixed-load throughput of 13.1 ops/s at 0% error rate; block finality of 2.40 s (p99 $<$ 3.05 s); issuance cost ≈\$0.00002[†]. **Limitations:** revocation requires O(n) Merkle rebuild; per-canister throughput is bounded by ICP's consensus serialization; no cross-chain support; prototype uses simplified hash functions and permissionless university registration requiring production hardening. Future work: W3C VC integration, multi-canister sharding, and accumulator-based O(1) revocation.

[†] Derived from ICP's official pricing formula [15] (updated Dec. 2025): ingress fee = 1.2 M + 2 K× bytes; update execution base = 5 M cycles; 1 executed Wasm instruction = 1 cycle; 1 T cycles = \$1.354820 USD. For a ≈400-byte `issueCertificate` payload, fixed fees total 7 M cycles. Code-complexity tracing of the Motoko source gives ≈100 K raw Wasm instructions: 2×`HashMap.get` (≈800), `createCertificate`/`bytesToHex` (≈33 K), `HashMap.put` (≈1.5 K), O($\log n$) `insertCertLeaf`/`combineHashes` (≈56 K), `CertifiedData.set` (≈3.2 K), metrics (≈3.3 K). Applying a 10–100× Motoko runtime overhead (GC, type-tag checks, bounds checks) yields 1–10 M instruction cycles. **Total: 8–17 M cycles ≈ \$0.000011–\$0.000023.**

References

[1] H. Chahal and S. Singh, "Credential fraud in South Asia: Findings and implications for the higher education sector," *Int. J. Educational Management*, vol. 35, no. 7, pp. 1498–1512, 2021.

[2] T. Hanke, M. Movahedi, and D. Williams, "DFINITY Technology Overview Series, Consensus System," *arXiv:1805.04548*, 2018.

[3] DFINITY Foundation, "The Internet Computer for Geeks," White Paper, 2022. [Online]. Available: https://internetcomputer.org/whitepaper.pdf

[4] V. Buterin, "A next-generation smart contract and decentralized application platform," *Ethereum White Paper*, 2014. [Online]. Available: https://ethereum.org/whitepaper

[5] R. C. Merkle, "A certified digital signature," in *CRYPTO'89*, LNCS vol. 435, pp. 218–238, 1989.

[6] D. Boneh, B. Lynn, and H. Shacham, "Short signatures from the Weil pairing," in *ASIACRYPT 2001*, LNCS vol. 2248, pp. 514–532, 2001.

[7] J. Grech and A. F. Camilleri, "Blockchain in Education," EUR 28778 EN, Publications Office of the EU, Luxembourg, 2017.

[8] M. Turkanović et al., "EduCTX: A blockchain-based higher education credit platform," *IEEE Access*, vol. 6, pp. 5112–5127, 2018.

[9] D. Lizcano et al., "Blockchain-based approach to create a model of trust in open and ubiquitous higher education," *J. Computing in Higher Education*, vol. 32, pp. 109–134, 2020.

[10] M. Sporny et al., "Verifiable Credentials Data Model v1.1," W3C Recommendation, Mar. 2022. [Online]. Available: https://www.w3.org/TR/vc-data-model/

[11] G. Wood, "Ethereum: A secure decentralised generalised transaction ledger," *Ethereum Yellow Paper*, 2014.

[12] V. Buterin et al., "Combining GHOST and Casper," *arXiv:2003.03052*, 2020.

[13] T. T. A. Dinh et al., "BLOCKBENCH: A framework for analyzing private blockchains," in *Proc. ACM SIGMOD*, 2017, pp. 1085–1100.

[14] E. Androulaki et al., "Hyperledger Fabric: A distributed operating system for permissioned blockchains," in *Proc. EuroSys'18*, 2018, pp. 1–15.

[15] DFINITY Foundation, "Gas Cost," *Internet Computer Documentation*, Dec. 2025. [Online]. Available: https://docs.internetcomputer.org/building-apps/essentials/gas-cost

[16] C. Allen, "The path to self-sovereign identity," *Life With Alacrity blog*, Apr. 2016. [Online]. Available: http://www.lifewithalacrity.com/2016/04/the-path-to-self-soverereign-identity.html